

Simulador de Gestor de Procesos

para Sistemas Operativos

Implementación en Python · Proyecto de Semestre

Materia	Sistemas Operativos
Institución	Universidad Autónoma de Tamaulipas
Facultad	Facultad de Ingeniería Tampico
Semestre	Sexto Semestre 2026
Profesor	Dr. Dante Adolfo Muñoz Quintero
Integrantes	César Olvera Rodríguez Brian Satihs Hernández González Jocelyin Mateo Saldierna
Fecha	Mayo 2026

Este documento presenta la Demostración Funcional del proyecto de semestre correspondiente a la materia de Sistemas Operativos.

En él se describe el diseño del sistema, se incluyen fragmentos del código fuente con sus respectivos comentarios y se muestran capturas de la ejecución del simulador.

Tabla de Contenidos

Tabla de Contenidos	2
1. Introducción	3
2. Objetivos	4
Objetivo general	4
Objetivos específicos.....	4
3. Diseño del sistema.....	5
3.1 Arquitectura general	5
3.2 Diagrama general de módulos / componentes.....	5
3.3 Flujo de datos entre módulos.....	6
3.4 Diagrama de componentes.....	7
3.5 Diagrama de estados del proceso (PCB).....	8
3.6 Decisiones de diseño relevantes	9
4. Implementación.....	10
4.1 Operaciones sobre procesos — core/proceso.py	10
4.2 Asignación de recursos — core/recursos.py.....	11
4.3 Algoritmos de planificación — algoritmos/	11
4.4 Comunicación y sincronización — ipc/.....	12
4.5 Terminación de procesos — core/scheduler.py	13
4.6 Interfaz gráfica — ui/menu.py	14
5. Demostración funcional.....	15
5.1 Creación, suspensión y terminación de procesos	15
5.2 Planificación con cada algoritmo	15
5.3 Caso productor-consumidor (IPC)	16
5.4 Logs de eventos del simulador	16
6. Conclusiones	17
Trabajo futuro	17
7. Referencias.....	18

1. Introducción

Los sistemas operativos modernos coordinan múltiples procesos al mismo tiempo, repartiendo CPU, memoria y otros recursos entre tareas que compiten por ellos. Entender cómo funciona ese mecanismo puede parecer abstracto cuando solo se estudia en papel; este proyecto existe precisamente para cambiar eso.

El Simulador de Gestor de Procesos es una aplicación de escritorio desarrollada en Python con interfaz gráfica tkinter. No ejecuta procesos reales del sistema operativo, sino que modela su comportamiento mediante objetos, colas, estados y eventos de simulación. La idea es que cualquier persona pueda crear procesos, elegir un algoritmo de planificación, avanzar la simulación tick a tick y observar en pantalla exactamente qué ocurre con la CPU, la memoria y la cola de listos.

El proyecto cubre los conceptos centrales de la materia: ciclo de vida de un proceso, algoritmos FCFS y Round Robin, gestión de memoria simulada, semáforos, el problema clásico del productor-consumidor y comunicación asíncrona por buzón de mensajes. Todo sin dependencias externas; solo Python y sus librerías estándar.

Este documento se organiza de la siguiente manera: la sección 2 presenta los objetivos del proyecto; la sección 3 describe el diseño del sistema; la sección 4 contiene fragmentos del código fuente comentado; la sección 5 muestra capturas de la ejecución del simulador; finalmente las secciones 6 y 7 presentan conclusiones y referencias.

2. Objetivos

Objetivo general

Desarrollar un simulador interactivo de gestor de procesos en Python que implemente los conceptos fundamentales de sistemas operativos vistos en clase, incluyendo operaciones sobre procesos, asignación y liberación de recursos, algoritmos de planificación, comunicación entre procesos y sincronización mediante semáforos.

Objetivos específicos

1. Implementar la estructura de datos PCB (Process Control Block) con PID autogenerated de forma secuencial, nombre, estado, burst time, tiempo restante, memoria requerida y causa de terminación.
2. Modelar las transiciones de estado del proceso: listo, ejecutando, esperando, suspendido y terminado, validando que solo se realicen cambios válidos.
3. Simular un entorno de hardware limitado con una CPU monoprocesador y 4096 MB de memoria RAM, gestionando solicitudes y liberaciones por proceso.
4. Implementar los algoritmos de planificación FCFS (no preemptivo) y Round Robin (preemptivo con quantum configurable).
5. Demostrar sincronización mediante el problema clásico del productor-consumidor, utilizando semáforos contadores y mutex sobre un buffer compartido de capacidad fija.
6. Implementar comunicación asíncrona entre procesos a través de un buzón de mensajes identificado por PID.
7. Registrar causas de terminación (normal, forzada por usuario) en un log de eventos con tick, tipo y mensaje.
8. Proveer una interfaz gráfica con tkinter que permita crear procesos, visualizar el estado de la CPU, la memoria y la cola de listos en tiempo real, y controlar la simulación de forma manual o automática.

3. Diseño del sistema

3.1 Arquitectura general

El simulador sigue una arquitectura modular organizada en capas. Cada capa agrupa módulos con una responsabilidad específica y se comunica con las demás a través de interfaces bien definidas. El archivo `main.py` es el punto de entrada; instancia la aplicación `tkinter` y cede el control a la interfaz gráfica, que actúa como orquestador central del sistema.

Capa	Módulo / Archivo	Responsabilidad
Entrada	<code>main.py</code>	Punto de entrada. Inicializa <code>tk.Tk()</code> y lanza la interfaz gráfica.
Entrada	<code>test_imports.py</code>	Valida que todos los módulos se importan correctamente.
Lógica central	<code>core/proceso.py</code>	PCB, máquina de estados, PID autoincrementado.
Lógica central	<code>core/scheduler.py</code>	Cola de listos, despacho por algoritmo, tick global, integración con IPC.
Lógica central	<code>core/recursos.py</code>	Pool de CPU y memoria, solicitud y liberación.
Algoritmos	<code>algoritmos/fcfs.py</code>	FCFS: no preemptivo, orden de llegada estricto.
Algoritmos	<code>algoritmos/round_robin.py</code>	Round Robin: preemptivo con quantum configurable.
IPC	<code>ipc/sincronizacion.py</code>	Semaforo (<code>wait/signal</code>) y ProductorConsumidor (buffer acotado).
IPC	<code>ipc/mensajes.py</code>	BuzonMensajes: comunicación asíncrona entre procesos.
Interfaz	<code>ui/menu.py</code>	Aplicación <code>tkinter</code> con paneles, controles y modo automático en hilo <code>daemon</code> .
Registro	<code>ui/logger.py</code>	Logger: almacena eventos con tick, tipo y mensaje.

Tabla 1. Módulos del simulador y sus responsabilidades.

3.2 Diagrama general de módulos / componentes

El diagrama siguiente muestra los módulos del subsistema de gestión de procesos en Python y sus relaciones de dependencia directa. La interfaz gráfica basada en `tkinter` (`ui/menu.py`) actúa como punto de entrada del usuario y orquestador central del sistema. Los módulos de `core/` y `algoritmos/` controlan el ciclo de vida, los estados y las políticas de planificación, mientras que los componentes de `ipc/` intervienen cuando los procesos necesitan comunicarse o sincronizarse. Todo el flujo de eventos queda registrado por `ui/logger.py`.

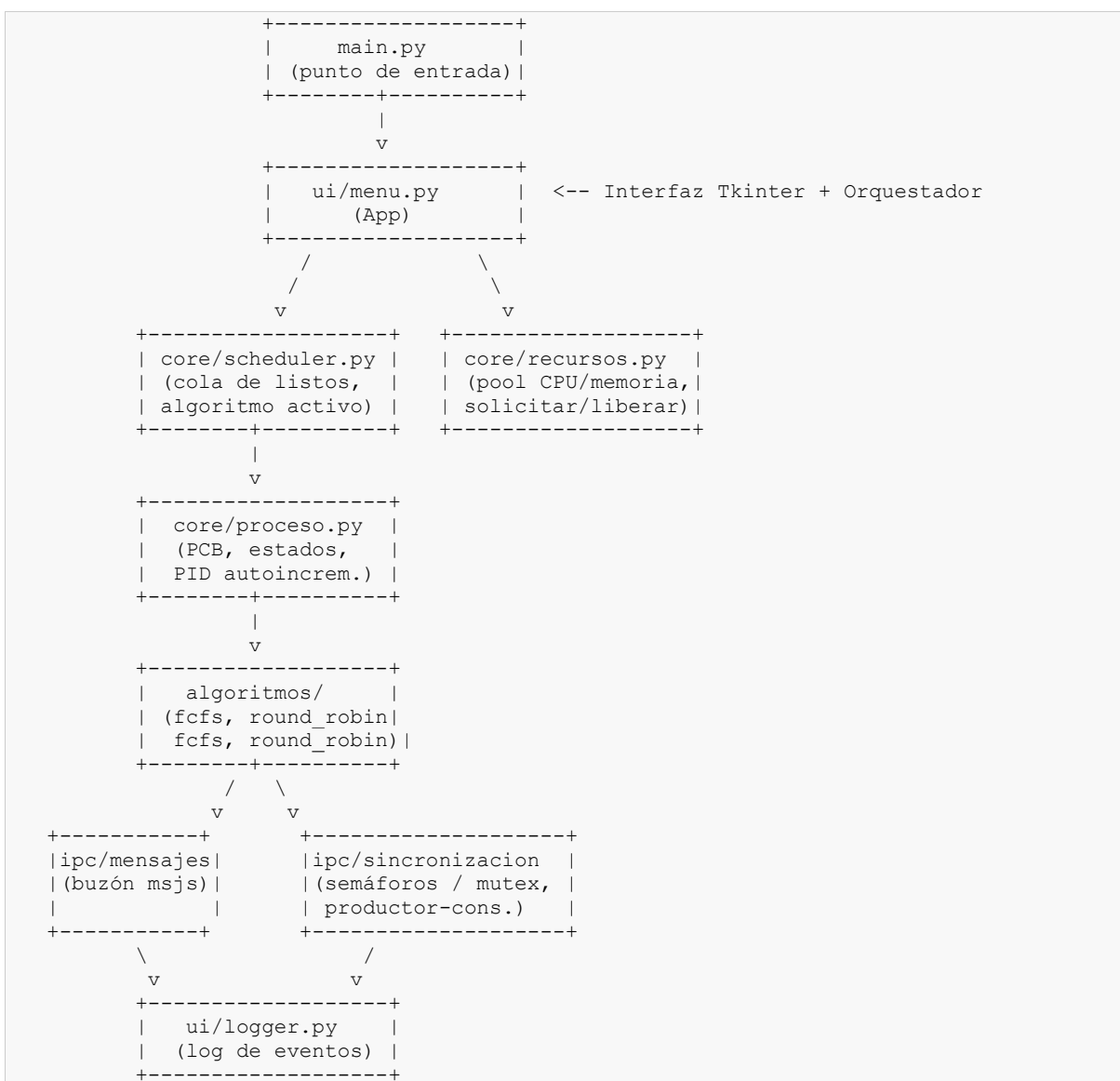


Figura 1. Relaciones de dependencia entre módulos del simulador. Las flechas indican «depende de / invoca».

3.3 Flujo de datos entre módulos

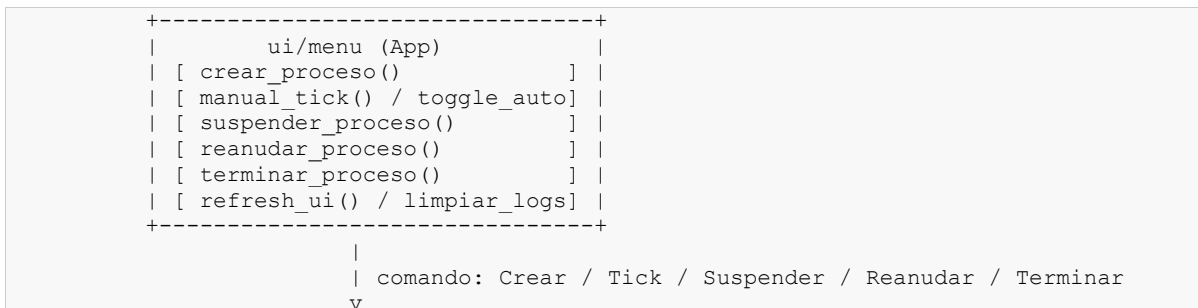
El siguiente diagrama describe cómo circula la información en el sistema durante el ciclo de vida de un proceso en el entorno Python/tkinter: desde que el usuario envía los parámetros a través del formulario, pasando por la validación de recursos, hasta el procesamiento por ticks de los algoritmos de planificación. Las etiquetas sobre las flechas indican el tipo de dato, la llamada a método o el mensaje intercambiado.



Figura 2. Flujo de datos desde la creación hasta la terminación de un proceso.

3.4 Diagrama de componentes

El diagrama de componentes amplía el diagrama general mostrando las interfaces (métodos públicos) provistas y requeridas por cada componente, así como las estructuras de datos y clases que cruzan sus fronteras. Cada componente corresponde a un módulo o archivo .py dentro de sus respectivos paquetes (core/, ipc/, ui/, algoritmos/).



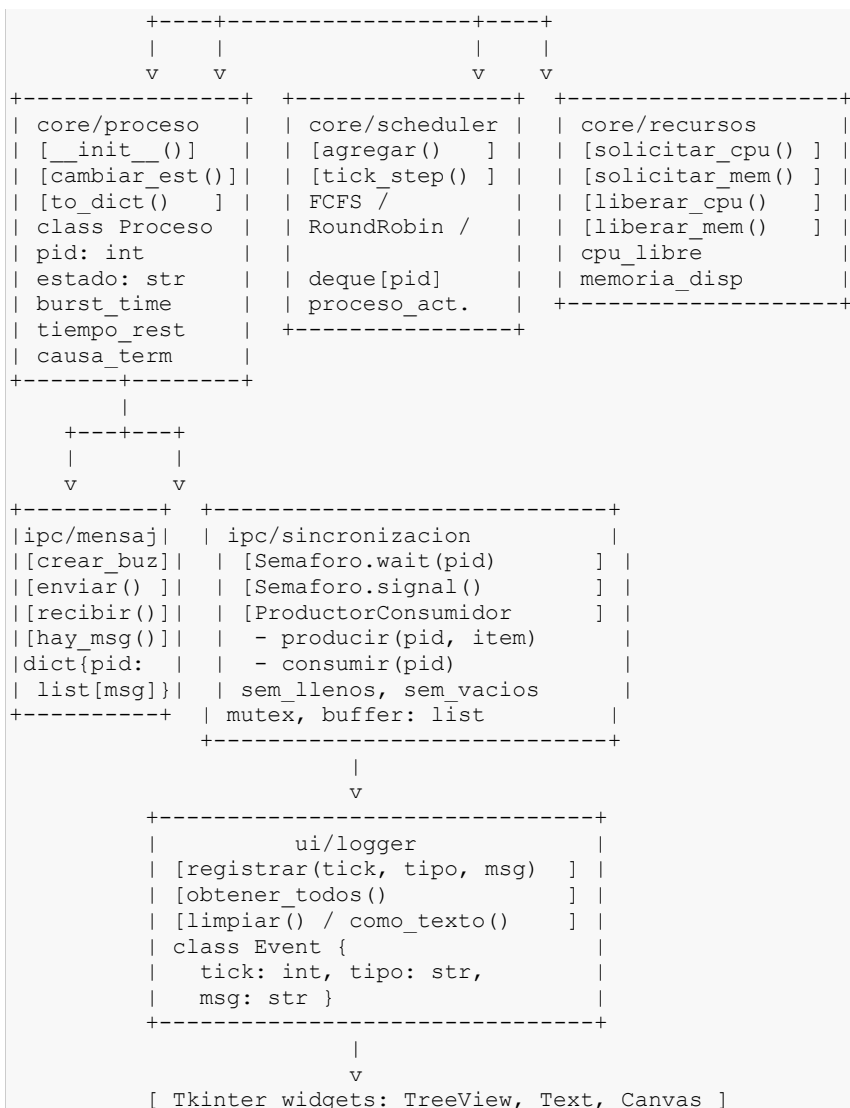
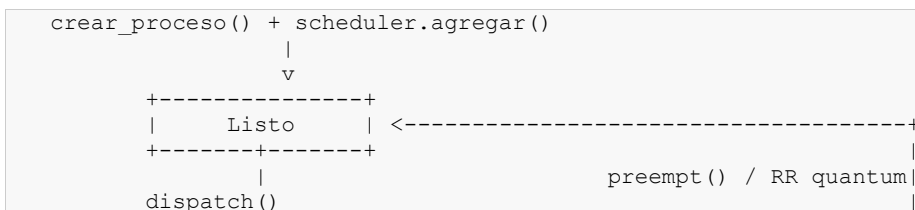


Figura 3. Diagrama de componentes con interfaces públicas y estructuras de datos relevantes de cada módulo.

3.5 Diagrama de estados del proceso (PCB)

Cada proceso es representado por una instancia de la clase Proceso en core/proceso.py y transita por diferentes estados lógicos a lo largo de su ciclo de vida. Las transiciones son coordinadas mediante el método `cambiar_estado()`, que valida que solo se realicen cambios definidos en el diccionario `ESTADOS_VALIDOS`. El flujo está dictado por el reloj del sistema (`tick_step()` en el planificador), el cual despacha los procesos, descuenta el burst time y gestiona las interrupciones o la suspensión manual desde los controles de la interfaz.



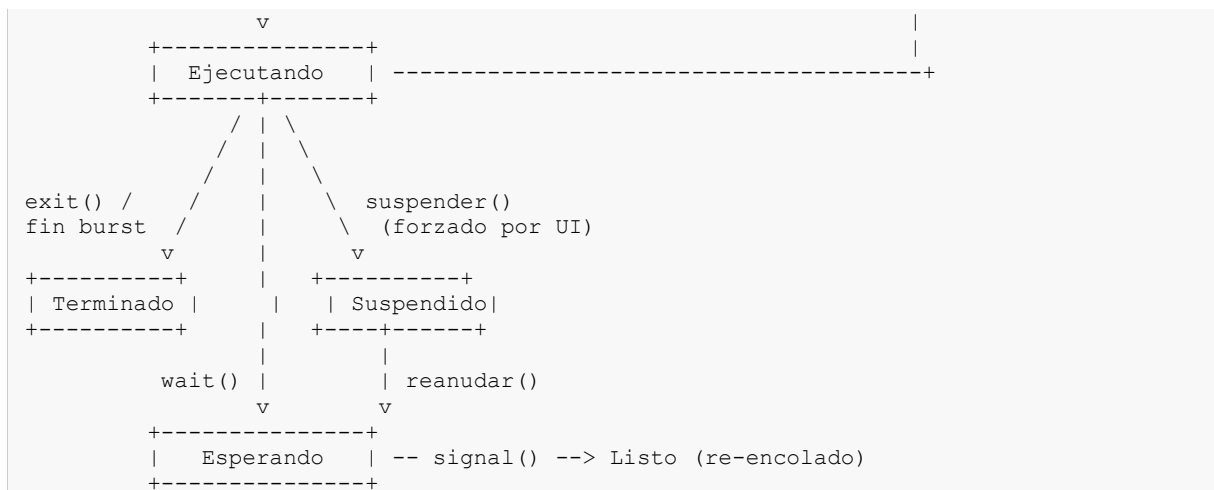


Figura 4. Transiciones válidas entre estados. El método `cambiar_estado()` rechaza cualquier cambio no definido en `ESTADOS_VALIDOS`.

3.6 Decisiones de diseño relevantes

- PID autoincrementado: se usa un contador de clase en Proceso que se incrementa con cada nueva instancia, garantizando unicidad sin necesidad de estructuras adicionales.
- Estado como cadena validada: el estado del proceso se almacena como string dentro de un conjunto `ESTADOS_VALIDOS`; `cambiar_estado()` rechaza cualquier valor fuera de ese conjunto, eliminando estados no contemplados.
- Cola de listos como deque: el uso de `collections.deque` permite insertar al final y extraer del frente en $O(1)$, lo que es natural tanto para FCFS como para Round Robin.
- Hilo daemon para el modo automático: el avance automático corre en un hilo separado marcado como daemon, lo que garantiza que se detenga limpiamente al cerrar la ventana principal sin necesidad de join explícito.
- Semáforo sobre contador y lista de bloqueados: el semáforo se implementa con un entero contador y una lista de PIDs bloqueados, lo que es suficiente para el modelo didáctico sin necesidad de primitivas del sistema operativo real.
- Logger desacoplado: el logger acumula eventos en una lista interna y la interfaz los consulta en cada ciclo de refresh, de modo que el núcleo nunca bloquea esperando actualizar la pantalla.

4. Implementación

A continuación se presentan los fragmentos de código fuente más relevantes de cada módulo del simulador. El código completo se encuentra en el repositorio del proyecto.

4.1 Operaciones sobre procesos — `core/proceso.py`

Define la clase `Proceso` y su diccionario de transiciones válidas. El método `cambiar_estado()` valida que el cambio sea legal antes de aplicarlo; si no lo es, lo ignora y registra un aviso.

`core/proceso.py` — clase `Proceso` y máquina de estados

```
class Proceso:
    _contador_pid = 1

    ESTADOS_VALIDOS = {
        'listo': ['ejecutando', 'suspendido'],
        'ejecutando': ['listo', 'terminado', 'suspendido', 'esperando'],
        'esperando': ['listo'],
        'suspendido': ['listo'],
        'terminado': [],
    }

    def __init__(self, nombre, burst_time, memoria_req, tiempo_llegada=0):
        self.pid = Proceso._contador_pid
        Proceso._contador_pid += 1
        self.nombre = nombre
        self.estado = 'listo'
        self.burst_time = burst_time
        self.tiempo_rest = burst_time
        self.memoria_req = memoria_req
        self.tiempo_llegada = tiempo_llegada
        self.causa_term = None
        self.recursos = []

    def cambiar_estado(self, nuevo_estado):
        permitidos = self.ESTADOS_VALIDOS.get(self.estado, [])
        if nuevo_estado in permitidos:
            self.estado = nuevo_estado
        # cambio inválido se ignora silenciosamente
```

4.2 Asignación de recursos — core/recursos.py

Modela un pool con 1 núcleo de CPU y 4096 MB de memoria. Las solicitudes verifican disponibilidad antes de asignar; si no hay suficiente, devuelven False y el planificador rechaza la operación.

core/recursos.py — pool de CPU y memoria

```
class GestorRecursos:
    def __init__(self):
        self.memoria_total = 4096 # MB
        self.memoria_disp = 4096
        self.cpu_libre = True
        self.pid_en_cpu = None

    def solicitar_memoria(self, pid, cantidad):
        if self.memoria_disp >= cantidad:
            self.memoria_disp -= cantidad
            return True
        return False

    def liberar_memoria(self, cantidad):
        self.memoria_disp = min(self.memoria_total,
                                self.memoria_disp + cantidad)

    def solicitar_cpu(self, pid):
        if self.cpu_libre:
            self.cpu_libre = False
            self.pid_en_cpu = pid
            return True
        return False

    def liberar_cpu(self):
        self.cpu_libre = True
        self.pid_en_cpu = None
```

4.3 Algoritmos de planificación — algoritmos/

El planificador delega cada tick al módulo de algoritmo activo. A continuación se muestran las funciones de despacho de FCFS y Round Robin.

algoritmos/fcfs.py — despacho FCFS

```
def fcfs_tick(scheduler):
    r = scheduler.recursos
    # Si no hay proceso en CPU, tomar el primero de la cola
    if r.cpu_libre and scheduler.cola_listos:
        pid = scheduler.cola_listos.popleft()
        proc = scheduler.procesos[pid]
        r.solicitar_cpu(pid)
        proc.cambiar_estado('ejecutando')
        scheduler._registrar_log(f'FCFS: PID {pid} toma CPU')

    # Decrementar tiempo del proceso activo
    if r.pid_en_cpu is not None:
        proc = scheduler.procesos[r.pid_en_cpu]
        proc.tiempo_rest -= 1
        if proc.tiempo_rest <= 0:
            scheduler.terminar_proceso(proc.pid, 'normal')
```

algoritmos/round_robin.py — despacho Round Robin

```
def rr_tick(scheduler):
    r = scheduler.recursos
    if r.cpu_libre and scheduler.cola_listos:
        pid = scheduler.cola_listos.popleft()
        proc = scheduler.procesos[pid]
        r.solicitar_cpu(pid)
        proc.cambiar_estado('ejecutando')
        scheduler.ticks_en_cpu = 0

    if r.pid_en_cpu is not None:
        proc = scheduler.procesos[r.pid_en_cpu]
        proc.tiempo_rest -= 1
        scheduler.ticks_en_cpu += 1

    if proc.tiempo_rest <= 0:
        scheduler.terminar_proceso(proc.pid, 'normal')
    elif scheduler.ticks_en_cpu >= scheduler.quantum:
        # Quantum agotado: desalojar y reencolar
        proc.cambiar_estado('listo')
        scheduler.cola_listos.append(proc.pid)
        r.liberar_cpu()
        scheduler._registrar_log(
            f'RR: PID {proc.pid} quantum agotado, regresa a cola')
```

4.4 Comunicación y sincronización — ipc/

El módulo ipc/sincronizacion.py implementa el semáforo contador y el problema del productor-consumidor. El módulo ipc/mensajes.py gestiona el buzón de mensajes entre procesos.

ipc/sincronizacion.py — semáforo contador

```
class Semaforo:
    def __init__(self, valor_inicial=1):
        self.valor = valor_inicial
        self.bloqueados = [] # lista de PIDs en espera

    def wait(self, pid=None):
        if self.valor > 0:
            self.valor -= 1
            return True
        else:
            if pid:
                self.bloqueados.append(pid)
            return False

    def signal(self):
        if self.bloqueados:
            pid = self.bloqueados.pop(0)
            return pid # PID desbloqueado
        else:
            self.valor += 1
            return None
```

ipc/sincronizacion.py — ProductorConsumidor

```

class ProductorConsumidor:
    def __init__(self, capacidad=5):
        self.buffer = []
        self.capacidad = capacidad
        self.sem_vacios = Semaforo(capacidad) # espacios libres
        self.sem_llenos = Semaforo(0)         # ítems disponibles
        self.mutex = Semaforo(1)              # acceso exclusivo

    def producir(self, pid, item):
        if not self.sem_vacios.wait(pid):
            return False # buffer lleno
        self.mutex.wait()
        self.buffer.append(item)
        self.mutex.signal()
        self.sem_llenos.signal()
        return True

    def consumir(self, pid):
        if not self.sem_llenos.wait(pid):
            return None # buffer vacío
        self.mutex.wait()
        item = self.buffer.pop(0)
        self.mutex.signal()
        self.sem_vacios.signal()
        return item

```

4.5 Terminación de procesos — core/scheduler.py

Al terminar un proceso se liberan sus recursos, se registra la causa y, si hay un buzón activo para el siguiente proceso en cola, se envía una notificación indicando que la CPU quedó libre.

core/scheduler.py — rutina de terminación

```

def terminar_proceso(self, pid, causa):
    proc = self.procesos.get(pid)
    if not proc:
        return
    proc.causa_term = causa
    proc.cambiar_estado('terminado')
    self.recursos.liberar_memoria(proc.memoria_req)
    self.recursos.liberar_cpu()
    self._registrar_log(
        f'PROCESO: PID {pid} terminado - causa: {causa}')
    # Notificar al siguiente proceso si hay buzón activo
    if self.cola_listos and self.buzon:
        siguiente = list(self.cola_listos)[0]
        self.buzon.enviar(
            origen=pid,
            destino=siguiente,
            texto=f'CPU libre - PID {pid} terminó ({causa})'
        )

```

4.6 Interfaz gráfica — ui/menu.py

La clase App centraliza todos los controles. A continuación se muestra la lógica del tick manual y del modo automático, que corre en un hilo daemon separado.

ui/menu.py — tick manual y modo automático

```
def manual_tick(self):
    self.scheduler.tick_step()
    self.refresh_ui()

def toggle_auto(self):
    self.auto_on = not self.auto_on
    self.btn_auto.config(
        text='Auto OFF' if self.auto_on else 'Auto ON')
    if self.auto_on:
        t = threading.Thread(target=self._auto_loop, daemon=True)
        t.start()

def _auto_loop(self):
    while self.auto_on:
        self.scheduler.tick_step()
        self.after(0, self.refresh_ui) # actualizar en hilo principal
        velocidad = self.slider_vel.get() # ms entre ticks
        time.sleep(velocidad / 1000)
```

5. Demostración funcional

Las siguientes capturas muestran la ejecución del simulador con diferentes configuraciones. Cada escenario fue ejecutado con Python 3.x en el entorno de desarrollo del equipo.

5.1 Creación, suspensión y terminación de procesos

Se inicia el simulador con el algoritmo FCFS. Se crean tres procesos con diferentes valores de burst time y memoria. El proceso P1 entra a la CPU al primer tick; P2 y P3 quedan en la cola de listos.

The screenshot shows the 'PROCESOS' and 'RECURSOS' panels. In the 'PROCESOS' panel, a table lists three processes:

PID	Nombre	Estado	Burst	Restante
1	P1	terminado	5	0
2	P2	listo	5	5
3	P3	listo	10	10

The 'RECURSOS' panel shows the CPU state as 'LIBRE' (Free) in a green box. The memory usage is 1352/4096 MB. The 'Cola de Listos' (Ready Queue) shows two processes: PID 2: P2 (restante: 5) and PID 3: P3 (restante: 10).

Captura 5.1 — Creación de tres procesos con FCFS activo y terminación normal de P1.

5.2 Planificación con cada algoritmo

Se cambia el algoritmo a Round Robin con quantum = 2. Los tres procesos se reparten la CPU en turnos. Al agotarse el quantum, el proceso activo vuelve al final de la cola de listos.

The screenshot shows the 'PROCESOS' and 'RECURSOS' panels. The 'Algoritmo' is set to 'RoundRobin' and 'Quantum' is 2. In the 'PROCESOS' panel, a table lists three processes:

PID	Nombre	Estado	Burst	Restante
1	P1	listo	5	3
2	P2	listo	8	6
3	P3	ejecutando	5	4

The 'RECURSOS' panel shows the CPU state as 'EN USO (PID 3)' (In Use by PID 3) in a red box. The memory usage is 1864/4096 MB. The 'Cola de Listos' (Ready Queue) shows two processes: PID 1: P1 (restante: 3) and PID 2: P2 (restante: 6).

Captura 5.2 — Round Robin con quantum=2: rotación de procesos en CPU.

5.3 Caso productor-consumidor (IPC)

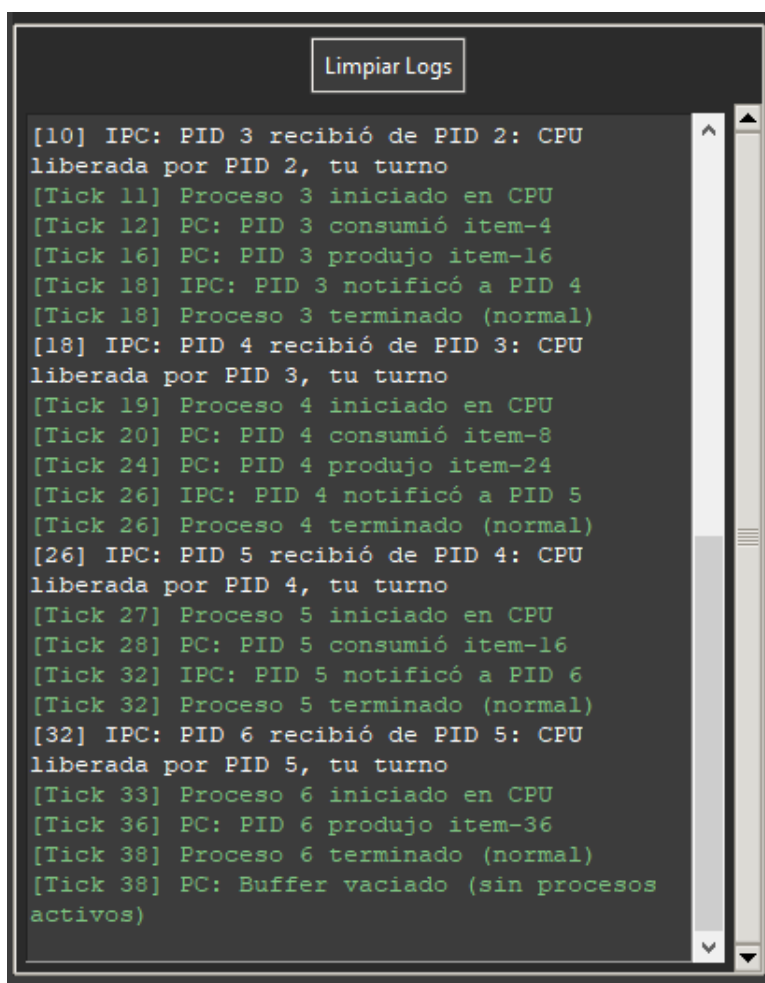
Con procesos activos en la cola de listos, el planificador activa el módulo IPC cada cuatro ticks. El buffer parte vacío; cuando hay espacio se produce un ítem y cuando está más cargado se consume.



Captura 5.3 — Actividad del buffer productor-consumidor cada 4 ticks.

5.4 Logs de eventos del simulador

El panel de logs registra cada evento con tick, tipo y mensaje. A continuación se muestra un fragmento del historial correspondiente a la sesión de las capturas anteriores.



Captura 5.4 — Log cronológico de eventos del simulador.

6. Conclusiones

César Olvera Rodríguez.

Antes de arrancar con esto asumí que implementar FCFS y Round Robin iba a ser la parte fácil, casi mecánica. Y sí, la lógica de cada algoritmo no es complicada por sí sola. Lo que no anticipé fue cuánto cambia el comportamiento del sistema completo según cuál uses. Con FCFS un proceso pesado simplemente congela la cola; los demás esperan y no hay nada que hacer. Con Round Robin el sistema se siente más justo, pero hay más cosas que pueden salir mal: el contador de ticks en CPU, el reencole al final, soltar la CPU en el momento correcto. Tener la interfaz corriendo en paralelo ayudó mucho para detectar esos errores, porque los veías pasar tick a tick en lugar de tener que leer un log al final.

Brian Satihs Hernández González.

El productor-consumidor me tomó más intentos de los que quisiera admitir. En papel, tres semáforos y un buffer suenan a algo que se resuelve en una tarde. En la práctica, el orden importa más de lo que parece: si haces signal antes de soltar el mutex, o si revisas sem_vacios en el momento equivocado, el buffer termina en un estado que no tiene sentido y no siempre es obvio por qué. Lo que me funcionó fue ir paso a paso, imprimir el estado del buffer después de cada operación y comparar con lo que debería pasar según la teoría. Al final el código quedó bastante limpio, pero el camino no lo fue.

Jocelyin Mateo Saldierna.

Tkinter tiene sus peculiaridades. La que más tiempo me costó fue entender que no puedes tocar widgets desde un hilo que no sea el principal, aunque técnicamente el código corra sin errores visibles al principio. El modo automático usa un hilo daemon y en algún momento empezamos a ver comportamientos raros en la interfaz: labels que no se actualizaban, la tabla que se congelaba. La solución fue `self.after(0, callback)`, que básicamente le dice a tkinter que encole la actualización para cuando pueda procesarla en su propio hilo. Una vez que lo entiendes es sencillo, pero llegar ahí sin conocer esa limitación de tkinter no es obvio.

Trabajo futuro

- Implementar detección de interbloqueo mediante un grafo de asignación de recursos.
- Agregar soporte para memoria paginada y visualización de marcos de página.
- Añadir métricas de rendimiento por algoritmo: tiempo de espera promedio, tiempo de retorno y throughput.
- Permitir configurar la capacidad de memoria y el número de posiciones del buffer desde la interfaz.

7. Referencias

- [1] Silberschatz, A., Galvin, P. B. y Gagne, G. (2018). Operating System Concepts (10.a ed.). Wiley.
- [2] Tanenbaum, A. S. y Bos, H. (2015). Modern Operating Systems (4.a ed.). Pearson.
- [3] Python Software Foundation. (2024). threading — Thread-based parallelism. Recuperado de <https://docs.python.org/3/library/threading.html>
- [4] Python Software Foundation. (2024). collections — Container datatypes. Recuperado de <https://docs.python.org/3/library/collections.html>
- [5] Python Software Foundation. (2024). tkinter — Python interface to Tcl/Tk. Recuperado de <https://docs.python.org/3/library/tkinter.html>
- [6] Downey, A. B. (2016). The Little Book of Semaphores (2.a ed.). Green Tea Press. Recuperado de <https://greenteapress.com/semaphores/>